

# Guía de Laboratorio Semana 3 – Práctica N° 3

**Tema:** Elaboración de programas utilizando Estructuras Repetitivas (While, For), ciclos anidados, herramientas y técnicas de programación (acumuladores, contadores, banderas) y ejecución paso a paso de un programa 1, 2.

## Parte 1: Ejecución Paso a Paso (Depuración en Thonny)

Para comprender cómo funcionan las estructuras repetitivas, es fundamental utilizar la herramienta de depuración de nuestro IDE (Thonny). Esto nos permitirá ver cómo cambian los valores de las variables en cada repetición (iteración).

- **Actividad 1:** En Thonny, activa la vista de variables yendo al menú **View** y seleccionando **Variables** 3, 4.
- Para ejecutar un programa paso a paso, en lugar de usar F5, presiona **Ctrl+F5**. Luego, utiliza **F6** para dar un paso grande (por líneas) o **F7** para un paso pequeño (evaluando cada parte de la expresión matemática o lógica) 4.

## Parte 2: Estructura Repetitiva Indeterminada (while)

El ciclo while ejecuta un bloque de instrucciones siempre y cuando una condición lógica se evalúe como verdadera (True) 5. Es ideal cuando no sabemos exactamente cuántas veces se repetirá el ciclo, como al validar datos ingresados por el usuario.

- **Actividad 2 (Validación de entrada):** Escribe el siguiente código y ejecútalo paso a paso (Ctrl+F5) ingresando primero un número negativo y luego uno positivo. Observa cómo el ciclo "atrapa" el error y no deja avanzar hasta obtener un dato válido 6, 7:

```
• x = float(input('Entra un número para hallar su raíz cuadrada: '))
• while x < 0:
•     x = float(input('El número debe ser positivo!! Intenta de nuevo: '))
• print('El número ingresado es correcto:', x)
```

## Parte 3: Estructura Repetitiva Determinada (for y range())

El ciclo for es ideal para recorrer elementos o ejecutar un bloque un número determinado de veces. En Python se utiliza fuertemente en combinación con la función predefinida range() 7, 8.

- La función range(inicio, fin, paso) genera una secuencia de números desde el *inicio* hasta el *fin* - 1 9.
- **Actividad 3:** Crea un programa que imprima los números del 1 al 5. Observa cómo la variable i toma un nuevo valor en cada iteración 9:

```
• for i in range(1, 6):
•     print(i)
```

## Parte 4: Herramientas y Técnicas de Programación

Para resolver problemas complejos dentro de los ciclos, utilizamos variables con roles específicos 10:

1. **Acumuladores:** Variable para ir guardando el resultado parcial de una operación (como sumas o multiplicaciones). Debe inicializarse antes del ciclo con el elemento neutro (0 para sumas, 1 para productos) 11.
2. **Contadores:** Un tipo especial de acumulador que se incrementa en un valor constante (generalmente 1) cada vez que ocurre un evento 12, 13.
3. **Banderas (Indicadores):** Una variable lógica (True o False) que sirve para recordar si un evento ha sucedido o no durante las iteraciones 13.

## Parte 5: Ejercicios de Desarrollo (Retos de Laboratorio)

Aplica los conocimientos adquiridos para desarrollar los siguientes programas completos:

**1. Suma de los primeros N números naturales (Acumulador y ciclo while o for):** Desarrolla un programa que solicite al usuario un número natural  $n$ . Utilizando un ciclo y un **acumulador**, calcula la suma de todos los números desde 1 hasta  $n$  y muestra el resultado 14. *(Pista: inicializa tu acumulador suma = 0 antes del ciclo y dentro del ciclo haz suma = suma + i)* 11, 14.

**2. Detectar números pares (Uso de Bandera y Contador):** Crea un programa que utilice un ciclo for para pedir al usuario exactamente 5 números enteros.

- Utiliza un **contador** para mostrar cuántos de esos números ingresados terminan en 7 13.
- Utiliza una **bandera** (inicializada en False) que cambie a True si el usuario ingresa algún número par. Al final del programa, imprime "Has introducido algún número par" o "Ningún número es par" dependiendo del estado de la bandera 13.

**3. Ciclos Anidados (Pares de elementos):** Desarrolla un programa que utilice dos ciclos for anidados para simular las fichas de un juego de dominó. El programa debe imprimir todas las combinaciones posibles desde (0, 0) hasta (6, 6) sin repetir combinaciones al revés 15. *(Pista: El ciclo exterior  $i$  va de 0 a 6, y el ciclo interior  $j$  debe empezar desde el valor actual de  $i$  hasta 6)* 15, 16.

*Nota para el docente: Valida que los estudiantes estén utilizando las teclas F6 y F7 para depurar los ciclos de la Parte 5. Es vital que vean cómo la variable índice ( $i$ ) del ciclo for se incrementa automáticamente, y cómo los contadores, acumuladores y banderas cambian de estado en la pestaña "Variables" de Thonny.* 3, 4, 17.